

Deliverable report

"Vittorio Polci": Mat: 257817, vittorio.polci@unitn.it, GitRepo: <https://github.com/vittorio01/GPU-Computing-2025-257817>

Abstract—This deliverable discusses about the implementation of different parallel solutions of the SpVM algorithm between CSR matrices and a general vector, with a final benchmark that compares all solution together.

Index Terms—Sparse Matrix, SpMV, CUDA, Parallelization, Storage Format

I. INTRODUCTION

The algorithm of sparse matrix-dense vector multiplication (SpMV) is an optimized version of the traditional matrix-vector multiplication designed to handle cases where the matrix is organized in a low density of non-zero entries. This technique is particularly useful in scenarios that involves very large sparse matrices because it allows to reduce the used space for storing that large structures and, at the same time, optimizes memory access patterns during computational steps.

Because the SpMV multiplication should handle matrices with millions or billions of not null elements, to obtain good results is necessary to use parallelization techniques on GPUs rather than sequential solution for CPUs.

II. PROBLEM STATEMENT

In the SpVM multiplication we consider a matrix coded in a sparse format (in this case the CSR format) and an input vector that has the same size of its columns number.

Each elements of the resulting vector is computed by performing the dot products of each not null elements of the matrix with the input vector and accumulating them in the corresponding row position. At this point the main objectives are:

- 1) Create a first sequential algorithm that performs the same operation on a sparse CSR matrix.
- 2) Parallelize the first CPU implementation with CUDA and compare them in terms of performances and resurces utilization.

A. Storage Format

The CSR for sparse matrices is a storage format that involves three different arrays to represent the information:

- The data array, that contains all not null terms of the original matrix.
- The column array, where for each element in the data vector specifies its column coordinate.
- The row array, which contains the start and the end pointers for each row to the data and column arrays. these two pair of pointers indicates which not null terms in the data array are contained in that specific row.

The size of the data and column arrays depends on the number of not null elements and the size of the row array depends on the number of the rows in the original matrix.

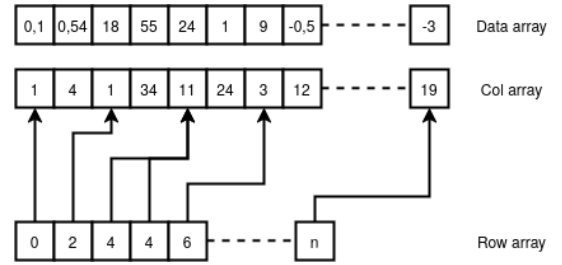


Fig. 1. Data organization in CSR structure

B. Parallelization

To improve the overall performance, the parallel implementations of the first algorithm are designed to be executed on Nvidia GPUs and programmed in CUDA. The CUDA language model permits to run a kernel on different concurrent threads called CUDA cores, where each thread represents a computational unit located in the GPU.

In general, the workload can be splitted in different ways for a certain number of CUDA cores and different organizations can provide more or less performance in terms of the execution time and troughput.

III. STATE OF THE ART

There already are multiple c++ libraries that performs CSR SpMV multiplications like:

- cuSPARSE [1], the official library supported by Nvidia that can perform multiplication between matrices and vectors in different formats with CUDA.
- Ginkgo [2], like MAGMA offers high-performance solution for multicore linear algebra algorithms for different platforms.

IV. METHODOLOGY AND CONTRIBUTIONS

A. Sequential algorithm

The most intuitive and efficient way to perform a sequential SpMV multiplication is to scan the row vector and, using the stored pointers, perform the product of the corredpondent not null elements and accumulate the result in a local variable.

The algorithm 1:

- 1) Scans the row array and decodes each pair of start and end pointes.
- 2) Using a second loop performs the product of each not null element with their correpondent elements in the

Algorithm 1 Sequential version for the SpVM algorithm

Require: Two vector structures *input* and *output* and a CSR matrix structure *matrix*.

```
1: procedure VMCSRML(output, matrix, input)
2:   for i in matrix.rowSize do
3:     startRow = matrix.rowArray[i]
4:     endRow = matrix.rowArray[i + 1]
5:     acc = 0
6:     for j in startRow, ..., endRow do
7:       col = matrix.colArray[j]
8:       acc +=
9:         matrix.dataArray[j] * input.dataArray[col]
10:    end for
11:    output.dataArray[i] = acc
12:  end for
13: end procedure
```

input vector and accumulates the results in the variable *acc*.

3) Stores the accumulated value in the output vector.

The memory access pattern of this solution permits coalescent accesses in all three arrays of the sparse matrix. The only aspect that limits the performance of the algorithm is the access pattern for gathering the input vector elements for the multiplication because it depends on the column coordinates stored in the column array of the matrix.

B. First parallel implementation

A first parallel implementation of the algorithm 1 divides the workload between threads such a way that each thread processes one or more entire rows

The first operation that the algorithm performs is divide the workload for each thread. In particular:

- 1) The row array is splitted in equal subsection that will be assigned to different blocks of threads using the ceiling division system.
- 2) The obtained subsection are then divided in the same way in order to assign equal workload to each thread.

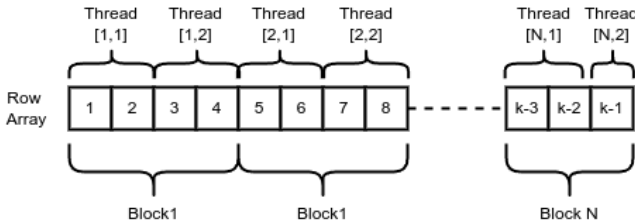


Fig. 2. Example of workload division for the first parallel implementation (N blocks of 2 threads)

After that procedure, each thread has his number of row to process *rowsPerThread*, the start position *threadStart* and the end position *threadEnd* in the row array.

The algorithm 2 is the simplest implementation and has good performances for common sparse matrices because:

Algorithm 2 First parallelization for the SpVM algorithm

Require: Same requirements in the sequential implementation but with all structures loaded in the global memory.

```
1: procedure VMCSRML(output, matrix, input)
2:   threadStart, threadEnd  $\triangleright$  Ceiling division process
3:   for row in threadStart, ..., threadEnd do
4:     startRow = matrix.rowArray[i]
5:     endRow = matrix.rowArray[i + 1]
6:     acc = 0
7:     for j in startRow, ..., endRow do
8:       col = matrix.colArray[j]
9:       acc +=
10:        matrix.dataArray[j] * input.dataArray[col]
11:    end for
12:    output.dataArray[row] = acc
13:  end for
14: end procedure
```

- For each thread the accesses are coalescent in all arrays of the matrix.
- Each thread works on the single rows independently and there is not concurrency at all.

C. Second parallel implementation

One problem of the algorithm 2 is that if there are threads that have much more more workload to perform respect others, the computational time might be affected negatively.

This is the case for matrices which have consecutive rows that contains much more not null elements compared to the others.

A solution that may mitigate this problem is to use a dynamic assignment of the rows to threads that are in the same block:

- 1) The row array is splitted in equal subsection for different blocks using again the ceiling division.
- 2) For each block a variable in the shared memory *pendingRow* is initialized and used to contain the next row number that is waiting for the assignment.
- 3) Each thread uses the function *atomicAdd(pendingRow, 1)* to increment the value of *pendingRow* and, at the same time, read its content
- 4) Using the value obtained each thread computes the corresponding row and repeats step 3 and 4 until the block reach the end of its subsection.

This dynamic assignment for threads in the same block implemented in the algorithm 3 tries to mitigate the unbalanced workload introduced in some rows. In facts, if some thread spend too much time when computing a single row, other available threads may perform also rows that in the algorithm 2 should be reserved for others.

Two possible limitation in terms of performance introduced with this system are that:

- Now there is concurrency for the value *pendingRow* that may limit the performance.

Algorithm 3 Second parallelization for the SpVM algorithm**Require:** Same requirements as the algorithm 2.

```

1: procedure VMCSRML(output, matrix, input)
2:   blockStart, blockEnd      ▷ Ceiling division
3:   pendingRow = 0            ▷ (only thread 0);
4:   syncthread()
5:   while true do
6:     pendingRow = atomicAdd(pendingRow, 1)
7:     +blockStart
8:     if pendingRow > blockEnd then return
9:   end if
10:  startRow = matrix.rowArray[pendingRow]
11:  endRow = matrix.rowArray[pendingRow + 1]
12:  acc = 0
13:  for j in startRow, ..., endRow do
14:    col = matrix.colArray[j]
15:    acc+ =
16:      matrix.dataArray[j]*input.dataArray[col]
17:  end for
18:  output.dataArray[pendingRow] = acc
19: end while
20: end procedure

```

- Threads may elaborate rows that are not organized in a coalescent way in the data and column arrays.

D. Third parallel implementation

Another tentative to distribute the workload contained in the single rows is to parallelize the algorithm such that each row is assigned to single blocks instead of threads:

- 1) The row array is splitted for different blocks as usual.
- 2) Each block initializes at 0 their corresponding part in the output vector.
- 3) For each row all threads does some multiplication (or not) of a certain number of elements and accumulates the result in the output vector using the function *atomicAdd()*.

The organization of algorithm 4 distributes the workload of a row on blocks of threads instead single threads without using shared memory optimization.

This situation may create more overhead because:

- If the block size is large the function *atomicAdd()* creates a barrier that limits too much the performance because it must guarantee atomic accumulation on the output vector.
- The first threads in a block are overused for all rows but the last threads are used only if the number of rows are large enough to require them.

V. SYSTEM DESCRIPTION AND EXPERIMENTAL SET-UP**A. Dataset description**

A dataset composed by 6 matrices is used for testing and benchmark all four algorithms. These matrices contains a number of not null elements between 1 and 1,5 million of

Algorithm 4 Second parallelization for the SpVM algorithm**Require:** Same requirements as the algorithm 2.

```

1: procedure VMCSRML(output, matrix, input)
2:   blockStart, blockEnd      ▷ Ceiling division
3:   ...                       ▷ output vector initialization phase
4:   for row in blockStart, ..., blockEnd do
5:     startRow = matrix.rowArray[row]
6:     endRow = matrix.rowArray[row + 1]
7:     elements = endRow - startRow
8:     if elements < blockSize.x then
9:       col = matrix.colArray[j]
10:      atomicAdd(
11:        output.dataArray[row],
12:        matrix.dataArray[threadIdx.x]*
13:        input.dataArray[col])
14:    else
15:      threadStart,
16:      threadEnd      ▷ Ceiling division on row
17:      acc = 0
18:      for j in threadStart, ..., threadEnd do
19:        col = matrix.colArray[j]
20:        acc+ = matrix.dataArray[j]*
21:          input.dataArray[col]
22:      end for
23:      atomicAdd(
24:        output.dataArray[row], acc)
25:    end if
26:  end for
27: end procedure

```

not null elements and they have different organization of the elements and dispersion that challenges the algorithms in different ways: The used dataset is obtained from the SuiteSparse Matrix Collection [3], a website that contains sample matrices which can be used for benchmarking algorithms.

Dataset	Dimension (R x C)	Non-zero entries	Type
dbir2 [4]	18,906 x 45,887	1,158,159	Unbalanced rows
ex11 [5]	16,614 x 16,614	1,096,948	Diagonally distributed (low density)
language [6]	399,130 x 199,130	1,216,334	Upper triangular
Linux_call_graph [7]	324,085 x 324,085	1,208,908	Dense matrix (worst case)
nemeth24 [8]	9,506 x 9,506	1,506,550	Diagonally distributed (best case)
twotone [9]	120,750 x 120,750	1,206,265	Diagonally distributed (moderate dispersion)

TABLE I
STRUCTURAL PATTERNS OF THE BENCHMARK SPARSE MATRICES.

B. System Description

The system used for the execution of the benchmarks is uses CUDA 12.3.2, GCC 11.5.0, and is composed by a Intel(R) Xeon(R) Silver 4214 CPU and A Nvidia A30 GPU.

C. Experimental Set-up

The performance is measured using the execution time elapsed measured with the C function *gettimeofday()* for the sequential algorithm and *cudaEventRecord* functions for CUDA algorithms.

Characteristic	Value	Device
Global memory	24 GB	GPU: Nvidia A30
Theoretical BW	933 GB/s	
CUDA Cores	10752	
SM Number	108	
FP32 Performance	10.3 TFLOPS	
Cores number	12	CPU: Intel(R) Xeon(R) Silver 4214
Threads number	24	
Clock speed	2.2Ghz	

TABLE II
BENCHMARK SYSTEM SPECS

Each algorithm is executed 25 times (5 warmup cycles and 20 iterations) and from the execution times (in milliseconds) is performed the geometric mean to remove eventual outliers.

From the average execution time is then performed the effective bandwidth and the FLOP/s.

$$effBW(GB/s) = \frac{readedBytes + writtenBytes}{executionTime(ms)} * 10^{-6} \quad (1)$$

$$GFLOPS/s = \frac{2 * notNull}{executionTime(ms)} * 10^{-6} \quad (2)$$

For all algorithm the number of byte transferred is equal to $(12 * notNull) + (12 * rowSize)$ for the first two algorithms and $(12 * notNull) + (16 * rowSize)$ for the third one.

Note that the effective bandwidth is calculated without considering the caching process during the execution. That may cause an overestimation of bytes that are actually transferred from or to the global memory.

VI. EXPERIMENTAL BENCHMARKS

The following tables shows the values of GFLOPs/s (III) and effective bandwidth (IV) obtained from benchmarking on 256 blocks of 512 using float values. The figure 3 gives also a visualization of table III.

Matrix	Parallel 1	Parallel 2	Parallel 3	Sequential
dbir2 [4]	152,93	142,78	34,8	1,69
ex11 [5]	71,44	71,74	21,02	0,38
language [6]	31	82,4	1,91	0,32
Linux_call_graph [7]	0,85	1,72	2,51	0,25
nemeth24 [8]	43,4	50	16,81	0,37
twotone [9]	44,42	51,84	4,7	0,42

TABLE III
GFLOPS/S COMPARISON BETWEEN PARALLEL AND SEQUENTIAL IMPLEMENTATIONS

Matrix	Parallel 1	Parallel 2	Parallel 3
dbir2 [4]	932,56	870,66	212,49
ex11 [5]	435,15	437	128,03
language [6]	250,37	656,65	15,27
Linux_call_graph [7]	6,52	13,11	19,12
nemeth24 [8]	263,8	305,51	102,18
twotone [9]	292,8	241,75	31,03

TABLE IV
BANDWIDTH (GB/S) COMPARISON BETWEEN PARALLEL IMPLEMENTATIONS

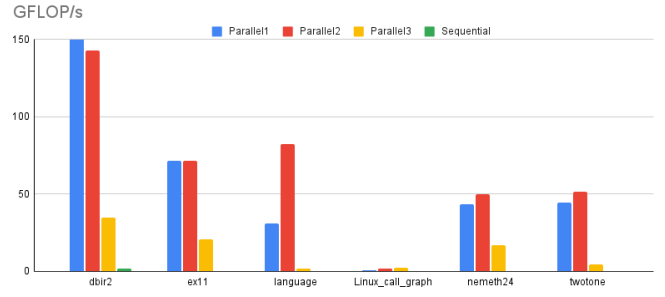


Fig. 3. Performance differences between the four algorithms

VII. RESULTS AND CONCLUSION

The results indicates that:

- The best situation is when the not null elements in the matrix are concentrated on the diagonal.
- If the number of not null elements per row is almost uniform for each row in the matrix, the first and the second parallel algorithms give similar results.
- The second parallel algorithm behaves better in different situations than the others implementations thanks to its load distribution system.
- The third parallel algorithm is more performant than the others only if the number of not null elements per row is very large and the dispersion of the matrix is high.

Also the block size affects the performances of the algorithms. In facts:

- In the first implementation the block size does not affect much the performances because the static allocation of workload.
- In the second implementation a lower block size may reduce the concurrency on the assignment system of the rows.
- In the third implementation a block size that is much lower than the average number of not null elements per row is more convenient because reduces significantly the concurrency introduced when accumulating the result on the output vector.

REFERENCES

- [1] NVIDIA. `cusparse`. [Online]. Available: <https://developer.nvidia.com/cusparse>
- [2] Ginkgo. [Online]. Available: <https://ginkgo-project.github.io>
- [3] U. of Florida. Suitesparse matrix collection. [Online]. Available: <https://sparse.tamu.edu/>
- [4] Meszaros. `dbir2` matrix. [Online]. Available: <https://sparse.tamu.edu/Meszaros/dbir2>
- [5] FIDAP. `ex11` matrix. [Online]. Available: <https://sparse.tamu.edu/FIDAP/ex11>
- [6] Tromble. `language` matrix. [Online]. Available: <https://sparse.tamu.edu/Tromble/language>
- [7] Sorensen. `Linux call graph` matrix. [Online]. Available: https://sparse.tamu.edu/Sorensen/Linux_call_graph
- [8] Nemeth. `nemeth24` matrix. [Online]. Available: <https://sparse.tamu.edu/Nemeth/nemeth24>
- [9] ATandT. `twotone` matrix. [Online]. Available: <https://sparse.tamu.edu/ATandT/twotone>